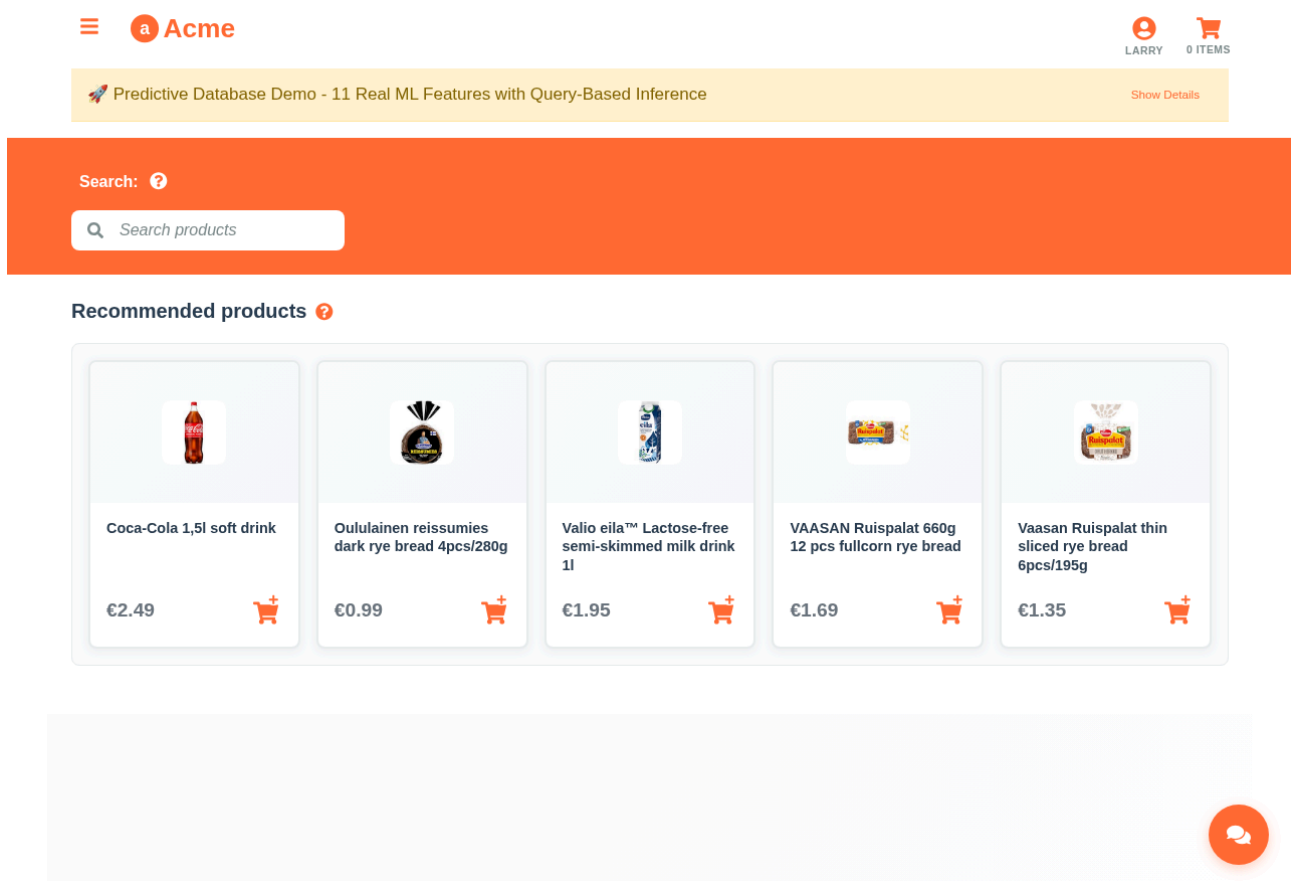


The Predictive Application

13 ways a predictive database transforms what software does.

A reference implementation across deliberately diverse application surfaces — personalised search, conversational UI, live analytics, NLP support, document classification, self-completing forms, predictive defaults, model self-evaluation. The Acme grocery setting is a coherence wrapper for the demo; the **point** is the breadth of application patterns one predictive database unlocks.



The Challenge — Intelligent Features Are Expensive

Modern applications want to feel intelligent. Search that personalises. Forms that fill themselves. Support that auto-routes. Dashboards that surface patterns. Automations that evaluate their own quality.

The traditional path to any one of those is an ML pipeline: data engineering, feature store, training infrastructure, deployment, drift monitoring, an owner. Most product teams can't justify that investment for a single feature — so features ship as static rules, and “intelligent” lives in the roadmap deck.



Pipelines per feature

Every smart capability needs its own model, its own training data, its own retraining schedule. Five features means five pipelines and five owners — most product teams stop at one.



Predictions outlive their training

The model trained on last quarter's data feels stale once the data distribution shifts. Nobody is watching the precision/recall curves until something visibly breaks.



Black-box decisions

A trained classifier returned answer X. Why? The product team has no recourse beyond “the model said so” — and the auditor has no recourse at all.

The Solution — Predictions Become Queries

Aito is a predictive database. Replace *train model* → *deploy service* → *monitor drift* with *SQL-like query* → *live prediction*. Every predictive feature you'd otherwise build as a separate ML service becomes a query against the same database that already stores your application data.

The 13 features in this demo are deliberately **not** a vertical product. They're 13 different application patterns — each a single query body, all sharing the same operators, the same explanations, the same evaluation discipline. The point isn't grocery retail. The point is what changes about software when intelligence is a database query rather than a project plan.



One technology, many patterns

Search, recommendation, classification, regression, relation analysis, evaluation — all share the same query API. Learn it once, ship it across the application.



Explainable by design

Every ``_predict`` returns the multiplicative chain that produced it — base rate × pattern lifts → final probability. Click any prediction to see why. No black box.



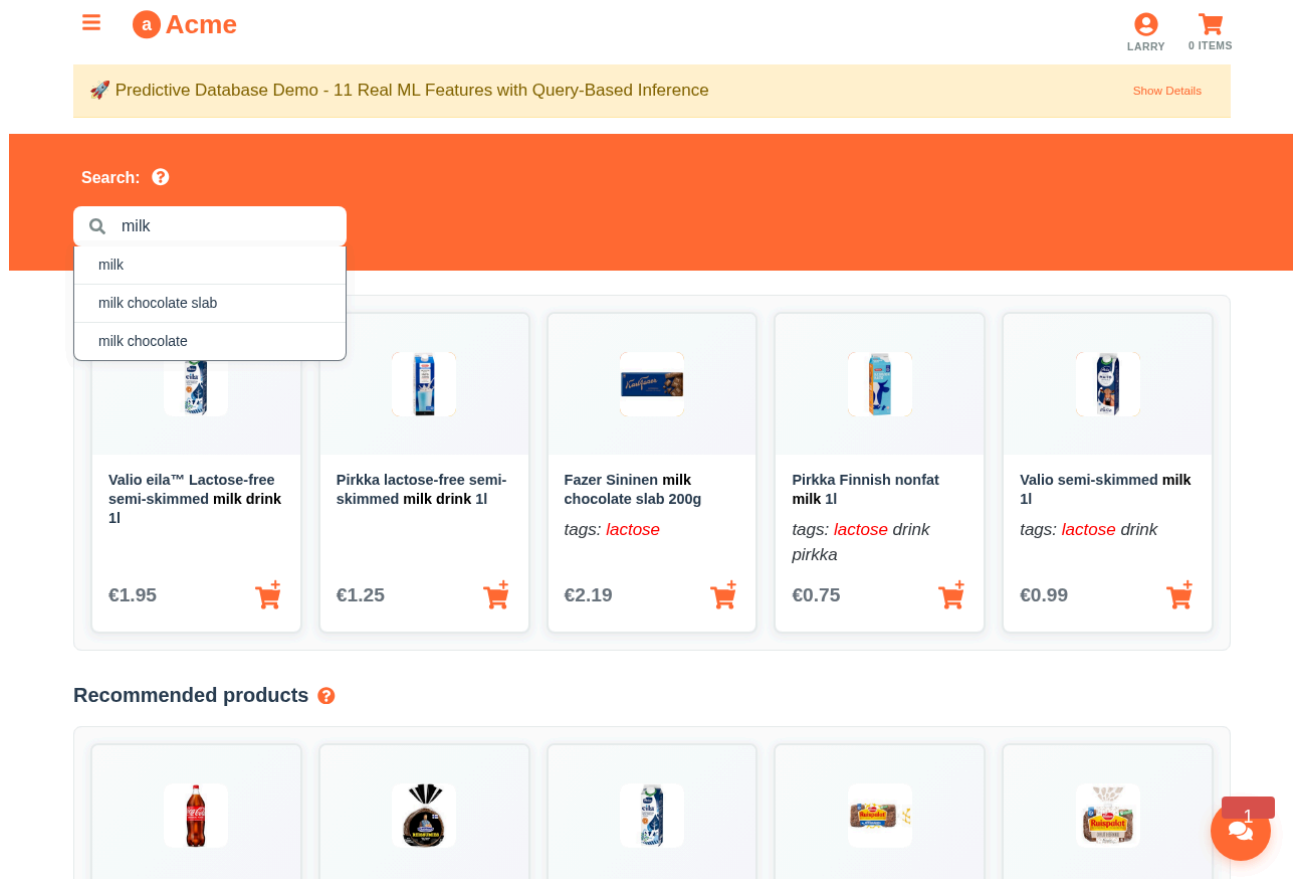
Honest when uncertain

``_evaluate`` reports accuracy AND accuracy gain. A 96 % model that doesn't beat the prior is flagged honest-failure, not shipped as success. The application knows when it doesn't know.

Pattern 1 — Personalised UI Surfaces

The application pattern: a list, grid, or feed that re-ranks per user without per-user models. The same query body returns a different ordering for every user, derived from the live interaction history in the database.

The Acme demo of this pattern is Smart Search. Type milk as a generic query. Switch the shopper pill between Larry (lactose- sensitive), Veronica (organic-only), and Alice (budget-driven) — three personas, three completely different top-5 lists, one query body, no retrained model in between.



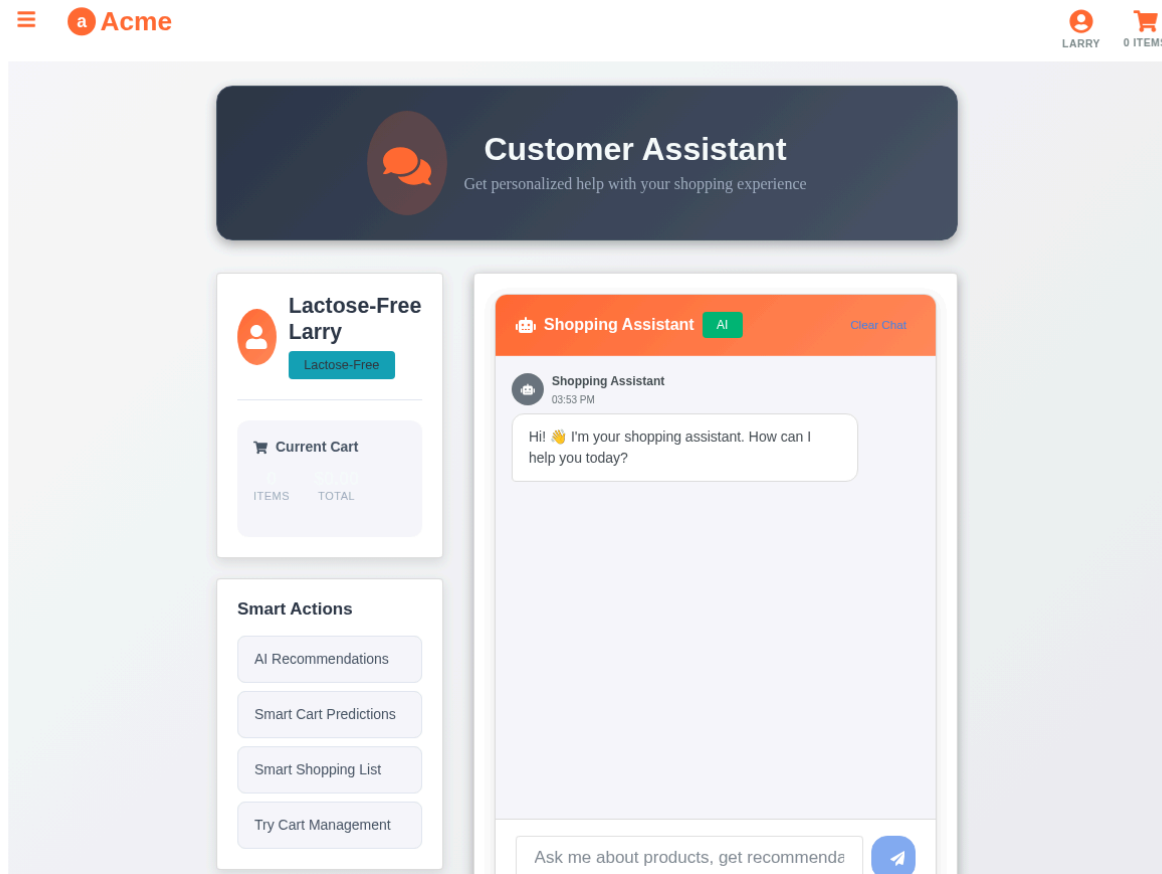
What's happening underneath:

- *Generic*: `_query` where `{product.name: {$match: "milk"}}` — token relevance only.
- *Predictive*: same `_query`, with `orderBy: $multiply($similarity, $p({purchase: true}))` and the active user pinned via `context.user`. Aito re-ranks the same rows by similarity-weighted purchase probability against the live history table.
- The only thing that varies between users is the user binding. The same pattern applies anywhere your application shows ranked content — search results, recommendations, notifications, feed ordering, support article suggestions.

Pattern 2 — Conversational Interfaces, Grounded

The application pattern: a chat surface where every answer is backed by a live query against your own application data, not a hallucinated sentence from training data. The LLM composes the prose; the predictive database supplies the facts.

The Acme demo is the Customer Assistant. “What’s good for a lactose- free dinner under €15?” — the assistant invokes `_recommend` and `_predict` as tool calls, then writes the reply. Same architecture also powers an Employee Assistant for back-office questions: one predictive database, two chat surfaces, different system prompts.



Three properties that distinguish predictive-database-backed chat from RAG-over-PDFs:

Tool-call grounded — every entity the model mentions came from a live Aito query in the same turn. No hallucinated SKUs, no stale knowledge cutoff.

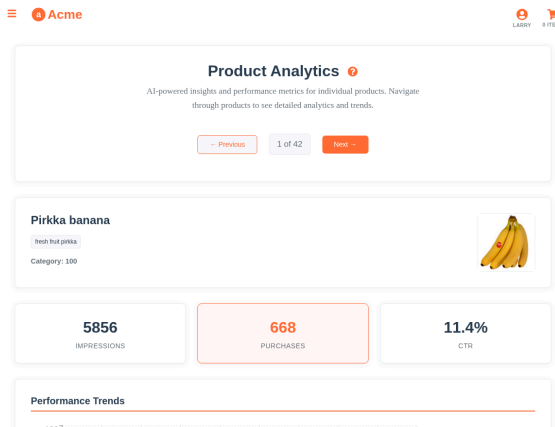
Context-aware — the active user (or session, or tenant) is part of the system prompt and conditions every `_recommend` goal issued under the chat.

Latency-visible — the LatencyPill above the chat reports the round-trip for each underlying Aito call. The user sees the answer cost one `_recommend` and one `_predict`, not a magic box.

Pattern 3 — Live Analytics Without Warehousing

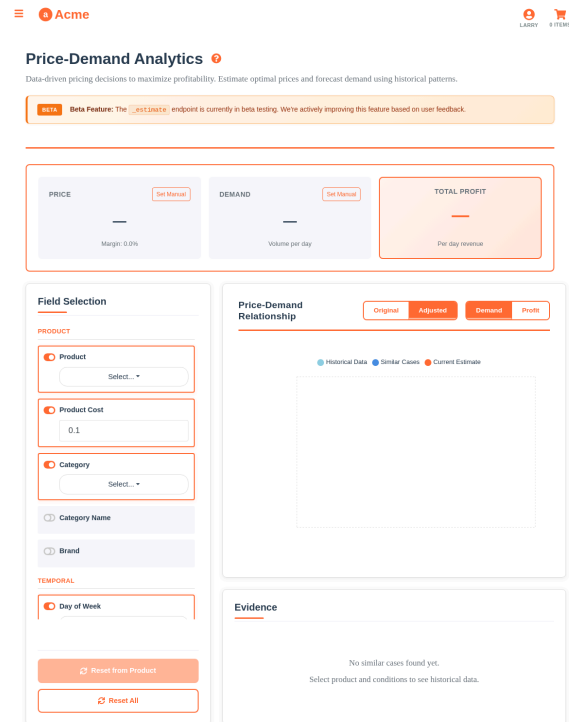
Per-Entity Dashboards

The application pattern: every chart in the analytics surface is a live query, not a nightly-aggregated rollup. Drill into a single entity — product, customer, supplier, ticket — and the panel populates from the same database the operational app reads.



Regression & Estimation

The application pattern: ask the data what a continuous variable should be — price, demand, latency, conversion — from a multivariate regression performed at query time. `_estimate` runs the regression on every call; no separately-trained price model.



Why this matters operationally: a product manager who has to choose between two intervention options answers the question against this week's data, not last quarter's snapshot. The same Aito instance that serves the operational application surfaces serves the analytics panels, so the numbers a user sees on a record page never disagree with the numbers an analyst sees on a dashboard.

The same pattern extends to **Preference Analytics** — `_relate` surfacing which segments correlate with which clusters. Demographic targeting that's mathematically defensible: every cell of the heatmap is a real lift number, derived from the data the application already holds.

Pattern 4 — Auto-Classification and Routing

The application pattern: documents, tickets, requests, or events arrive unstructured; the application labels them and routes them. Each routing decision is a `_predict` call that returns a class plus a confidence chip, with `$why` exposed for audit.

The Acme demo is Invoice Processing. Drop an invoice → GL code + approver + processing path suggested with confidence. The same shape generalises to support-ticket routing, expense categorisation, content moderation, fraud-flag triage, or any classify-and-route flow in the back office.

The screenshot shows the Acme Invoice Processing application. At the top, there's a navigation bar with the Acme logo and a shopping cart icon labeled 'LARRY' and '0 ITEMS'. The main header is 'Invoice Processing' with a help icon. Below it, a subtitle reads: 'Automatically classify and route invoices using AI-powered predictions. Enter invoice details to see intelligent suggestions for processor assignment, approval routing, and GL code classification.' There are two buttons: '+ New Invoice' and '< New Invoice >'. The interface is split into two columns. The left column, 'Invoice Details', has input fields for 'Invoice ID', 'Sender Name', 'Product Name', 'Account Number', and 'Description'. The right column, 'AI Predictions', shows suggestions for 'Processor' (David Green (Store Manager)), 'Acceptor' (Alice Johnson (COO)), and 'GL Code' (Grocery Store Products), each with a confidence chip and a help icon.

What this catches:

- **Routine items** — recurring patterns auto-route to the right owner with ≥ 95 % confidence and never touch a human
- **Anomalies** — out-of-pattern features drop confidence below threshold and route to manual review automatically — the application is correctly admitting it isn't sure
- **Audit-ready** — every prediction carries its `$why` chain. Compliance can answer “why did this go to Pekka?” by clicking the prediction, without scheduling a meeting

The demo's invoice schema is deliberately small (100 invoices, 10 GL codes, 10 employees). The query body doesn't change at production scale; Aito's prediction performance is independent of total row count up to multi-million rows.

Pattern 5 — Self-Completing Forms

The application pattern: a form where most fields fill themselves in from the few the user enters. Each suggested field is its own `_predict` or `_estimate` call against the application's own history; confidence chips block low-quality guesses; \$why opens the decision tree on demand.

The Acme demo is Product Catalog Enrichment. Paste a new SKU name → category, tags, and suggested price predicted in parallel. The same pattern is the spine of any application where data entry is high-friction: CRM enrichment, ticket triage forms, expense report classification, partner-onboarding intake forms.



Product Catalog

Manage products with AI-powered assistance. Enter a product name to automatically predict tags, category, and price based on similar products.

Add New Product

Product Name

Product Tags ?
Click on suggested tags below to add them:

gluten bread

Category AI Predicted (83% Confidence)

Price (€) AI Predicted (13% Confidence)

Add Product

What this catches:

- **Missing categorical fields** — “Vegan Oat Drink 1L” tokenises to tags: `vegan`, `plant-based`, `dairy-alternative` with > 90 % confidence
- **Missing continuous fields** — `_estimate` against history lands on a regressed price with a confidence band, not a point guess pretending to certainty
- **Locked attributes** — fields already supplied are predicted **anyway** and tagged 🔒 stored for honesty. Aito's prediction agrees with the stored value? Trust the catalog. Disagrees? Flag for review.

The Text type on name is load-bearing here — Aito tokenises and uses individual words as features. A schema with name: `String` would collapse this whole pattern to exact-name lookup and the moment would disappear. The detail matters anywhere this pattern lands.

Pattern 6 — Self-Evaluating Applications

The application pattern: every predictive surface in the app ships with a held-out `_evaluate` panel that reports two numbers — accuracy and accuracy **gain over the prior**. Predictions that don’t beat the naïve baseline are flagged in the application UI, not silently shipped. The application is honest about when it knows and when it doesn’t.

Acme

LARRY0 ITEMS

Model Quality

Monitor and evaluate AI model performance with real-time accuracy metrics and case analysis

Invoice Processing

Evaluation Configuration

Prediction Target

Invoice Processor

Choose what field to predict and evaluate

Data Split Configuration

90%10%

Train: 90% (random) | Test: 10% (random)

Input Fields

☒Description

☐SenderName

☐ProductName

☐AccountNumber

☐InvoiceID

Evaluate

Model Performance Metrics

100.00%

ACCURACY

0.00

MEAN RANK

14ms

AVG RESPONSE TIME

10

TEST SAMPLES

91

TRAINING SAMPLES

10 / 10

CORRECT PREDICTIONS

Evaluation Cases

STATUS	ID	SENDER	PRODUCT	ACCOUNT	DESCRIPTION
✓	INV-8004	Wu, Hall and ...	Grocery Store...	75392635	Purchase of Grocery ...
✓	INV-8014	Ross PLC	Grocery Store...	18980922	Purchase of Grocery ...
✓	INV-8022	Howard-Frost	Grocery Store...	36919250	Purchase of Grocery ...
✓	INV-8029	Banks-Miller	Grocery Store...	52081159	Purchase of Grocery ...
✓	INV-8047	Perez-Powers	Grocery Store...	76335772	Purchase of Grocery ...
✓	INV-8063	Walton-Weiss	ECommerce ...	71574090	Purchase of EComme...
✓	INV-	Eaton and Sone	Accounting S	45037302	Purchase of Accounti

The “accuracy” column is the easy metric. The “gain” column is the honest one — accuracy above what guessing the most common class would give you. If about 3 % of items get returned regardless of features, predicting “won’t be returned” for everything is 97 % accurate and adds zero information. That’s what +0 pp gain means.

Why this pattern is the most trust-building one in the demo: every predictive feature ships with its own evaluation, surfaced in the application, not buried in a notebook. Aito tells you when its predictions are real signal versus a coin flip dressed as accuracy. The naïve-metric trap is the one most products hide. A predictive application built this way labels it FAIL.

The seventh related pattern, **Predictive Defaults**, applies the same discipline to the form layer: the shopping cart pre-populates with the user's most-likely next basket, computed by `_recommend` against their visit history. Zero clicks to a usable default — and zero shipping of a prediction the evaluation panel can't defend.

How It Works



1. Connect your data

Whatever tables already power your application: products, users, events, documents, invoices, tickets. JSON schema declared up front. Single-shot upload, no separate feature store.



2. Query for predictions

Six operators cover the application patterns in this demo:

- `_predict`: classification
- `_recommend`: ranking & defaults
- `_relate`: correlation analysis
- `_query`: KPIs + analytics
- `_estimate`: regression
- `_evaluate`: honest pass/fail



3. Integrate

REST API · sub-100 ms response time · works from any application stack. This demo's frontend is React, but the same query bodies drop into mobile, server-side rendering, or back-office tools.

Architecture at a glance

Frontend — React 18 (Create React App) · Reactstrap + Bootstrap 5 · 13 numbered modules (`src/01-recommend.js` through `src/13-product-predictions.js`), each a standalone tutorial for one application pattern

Aito — REST API · `_predict` / `_recommend` / `_relate` / `_query` / `_estimate` / `_evaluate`

Data — 11 JSON tables shipped in `src/data/`, uploaded with `npm run upload-data`

Demo persona — 3 named users (Larry · Veronica · Alice) plus 67 tag-segmented users; menu organised by application-pattern pillar: Customer Experience · Analytics · Assistance · Automation

Coherence wrapper — the Acme grocery store is the scenario the 13 patterns share so the demo doesn't feel like 13 unrelated apps. The patterns are the deliverable; the grocery is the stage.

Deployment — Live at `demo.aito.ai` (Azure Web Apps); Netlify · Vercel · Docker · nginx configs all included

Designed to be read

Each of the 13 numbered modules in `src/` is a standalone tutorial for one Aito capability. Comprehensive JSDoc explains the application pattern and the business motivation. Use-case guides live in `docs/use-cases/`, one per module, with the full query body, expected response, and a “production checklist” for hardening before you ship the pattern in your own application.

What This Demo Isn't

This is a **deliberately broad** reference for predictive applications. The 13 features are not a coherent product — they're a cross-section of application patterns chosen for variety. We picked breadth over vertical depth on purpose. For depth in a specific vertical, look at the sibling demos:

Not an e-commerce platform — vertical e-commerce (smart search, recommendations, catalog enrichment, basket lift, return-risk evaluation) is the focus of `aito-ecommerce-demo`. If your roadmap is an online shop, that's the more relevant reference.

Not an ERP system — vertical ERP (goods receipt prediction, supplier matching, three-way match, inventory forecasting) is `aito-erp-demo`.

Not an accounting system — vertical accounting (ledger entry classification, invoice approval routing, financial flow automation) is `aito-accounting-demo`.

Not a complete grocery app — no checkout, payments, fulfilment, inventory, supply chain. Acme exists to give the 13 unrelated patterns a shared cast of characters. The grocery context is cosmetic; the patterns transfer.

Not personalised at scale — 70 demo users, 42 demo SKUs. Enough to show the patterns; not enough for per-individual modelling at production scale. The use-case docs note the data-volume thresholds where per-user `_recommend` starts paying off.

Not a UI showcase — the Reactstrap + Bootstrap UI is intentionally plain. Look at the queries, not the components. Production teams swap their own design system on top of the same query bodies.

Owning the scope is more credible than papering over it. The four Aito open-source demos (`aito-demo` · `aito-accounting-demo` · `aito-ecommerce-demo` · `aito-erp-demo`) deliberately partition into one cross-cutting **pattern reference** (this one) and three **vertical references** (the others). If you're picking which one to fork, this one is the right starting point when you want to see **what's possible**; the verticals are right when you know **what you're building**.

Ready to Build Predictive Applications?



See it live

demo.aito.ai — full 13-pattern tour, no signup. Switch between Larry, Veronica, and Alice to watch the predictive surfaces re-rank in real time.



Run it locally

git clone, npm install, npm start. The `.env.example` ships with a working public demo API key — predictions running in under 5 minutes from clone.



Talk to us

If your application roadmap has predictive search, conversation, classification, analytics, or evaluation on it, we should talk. EU hosted, no PII stored.

Build predictive applications — without the ML pipeline

Aito.ai is a predictive database. Upload the data your application already holds, query for predictions, ship features. The 13 patterns in this demo are the same query API you'd use in production — sub-100 ms `_predict` calls, full `$why` explanations, and honest `_evaluate` built in.

hello@aito.ai · aito.ai · github.com/AitoDotAI/aito-demo · demo.aito.ai

Aito.ai builds predictive infrastructure for product teams who want statistical intelligence without standing up an ML team. Open-source demos: aito-demo (this one — cross-cutting patterns) · aito-ecommerce-demo · aito-accounting-demo · aito-erp-demo.